VERIFIN 2.0

Interpretable hallucination detection for financial language models

Project report

November 2026

Abstract

Language models now write summaries and answers about financial reports, but they sometimes state figures, periods or facts that the report does not support. A single wrong number in an analyst's note can be costly, and a fluent paragraph gives no hint of which sentence to doubt. VERIFIN 2.0 is a web application that checks such statements against the source document. It splits a statement into atomic claims, retrieves the best evidence from the report with a hybrid of dense and keyword search, scores each claim with a natural-language-inference (NLI) model, and applies deterministic checks for figures, named entities, fiscal periods and table values. A fixed-order policy then returns one of three verdicts, *supported*, *contradicted* or *unverified*, together with the evidence page and the name of the rule that decided it.

On a held-out test split of 240 claims, scored once after the rules were frozen, VERIFIN reaches a macro-F1 of 95.2 (95% interval 92.3 to 97.7) and an accuracy of 94.6%. The NLI model alone scores 66.8, and a 7-billion-parameter language model given the same retrieved evidence scores 82.2 while taking about twelve times longer per claim. The result is that most of the accuracy comes from the deterministic layer rather than from a larger model. The system runs on a CPU at about one second per claim, calls no language model during verification, and records every action in a tamper-evident audit trail. The report also states what the system does badly: tables and statement layouts, wrong-direction claims whose figures are right, and a narrow definition of "unverifiable".

Contents

Abstract 2

1. Introduction 4

1.1 The problem . . . 4

1.2 Objectives . . . 4

1.3 Scope . . . 4

2. Approach and design choices 4

3. System architecture 5

3.1 The verification pipeline . . . 5

3.2 Service layout . . . 6

4. Features built around the verifier 6

4.1 Accounts and access . . . 6

4.2 Evidence and review . . . 6

4.3 Audit trail and provenance . . . 7

4.4 Analytics and benchmark pages . . . 7

5. Walkthrough 7

6. Evaluation 11

6.1 Data and method . . . 11

6.2 Held-out results . . . 11

6.3 Where the errors are . . . 12

6.4 What produced the gain . . . 13

6.5 A table-heavy probe . . . 14

7. Security and privacy 14

8. Testing and engineering practice 15

9. Limitations and responsible use 15

10. Future work 15

11. Conclusion 16

Appendix A. Main API endpoints 17

Appendix B. Reproducing the results 17

Appendix C. Notes on the figures 17

1. Introduction

1.1 The problem

Analysts, auditors and investors increasingly use language models to summarise annual reports, earnings calls and filings. These models are fluent, which makes their mistakes hard to see. In finance the typical mistakes are small and specific: a figure with a shifted decimal, a number quoted in thousands that the report gives in millions, last year's value presented as this year's, an increase described as a decrease, or a company, loan or acquisition that does not appear in the document at all. Each of these reads as plausible.

A useful checker has to do more than give a score. A reviewer needs to know which claim is doubtful, what the report actually says, on which page, and why the system decided as it did. Without that, a checker is one more opinion to second-guess.

1.2 Objectives

- Decide, for each claim in a statement, whether the source document supports it, contradicts it, or does not say.
- Show the evidence page and the deciding rule for every verdict, so a reviewer can verify the verdict instead of trusting it.
- Measure the system honestly on held-out data against simpler methods and a language-model judge, and report where it fails.
- Run on ordinary hardware (CPU only) using free tools, with no document text leaving the machine during verification.
- Provide what a team needs to use it: sign-in, organisations, roles, an audit trail, reports, analytics and an API.

1.3 Scope

The system verifies claims about one financial report at a time, in English. It is a screening aid for a human reviewer. It judges checkable facts and figures; it does not judge opinions, forecasts or causal explanations. Section 9 lists the limits in detail.

2. Approach and design choices

Three ways of building such a checker were considered, and the measurements in Section 6 compare them directly.

Approach	Strength	Weakness found here
Ask a language model to judge each claim	Flexible; reads natural language	Slow on a CPU (about 12 s per claim), and gets only 50% of period swaps and 33% of unit swaps right on test
Use an NLI model alone	Fast; trained for entailment and contradiction	Unreliable on numbers; 66.8 macro-F1 on test and cannot recognise a name that is absent from the report
Match the claim's numbers against the evidence	Simple and fast	Reaches 79.7 macro-F1 only because most errors in the test set are numeric; scores 0% on direction flips and unit swaps
VERIFIN: retrieval + NLI + deterministic checks + a verdict policy	Reproducible, traceable, fast; each verdict names its rule	Rules are tuned to the error types seen; tables and new layouts need their own checks

The central design decision is to keep a language model out of the verification path. Verdicts must be reproducible (the same input gives the same verdict) and traceable (each verdict names the rule behind it). A language-model judge gives neither, and the measurements show it is also less accurate and much slower on this task. A small NLI model is still useful, but as one input among several: the deterministic checks catch the figure, name and period errors that it misses, and the verdict policy decides in a fixed order.

3. System architecture

The backend is a FastAPI service in Python; the frontend is a React 19 and TypeScript application built with Vite, Tailwind CSS and Recharts. Documents, chunks, sessions, users and the audit chain are stored in MongoDB (Atlas in deployment, with an in-memory fallback for local development). The models are held in memory: MiniLM sentence embeddings, BM25 keyword ranking, and the DeBERTa-v3-small NLI cross-encoder.

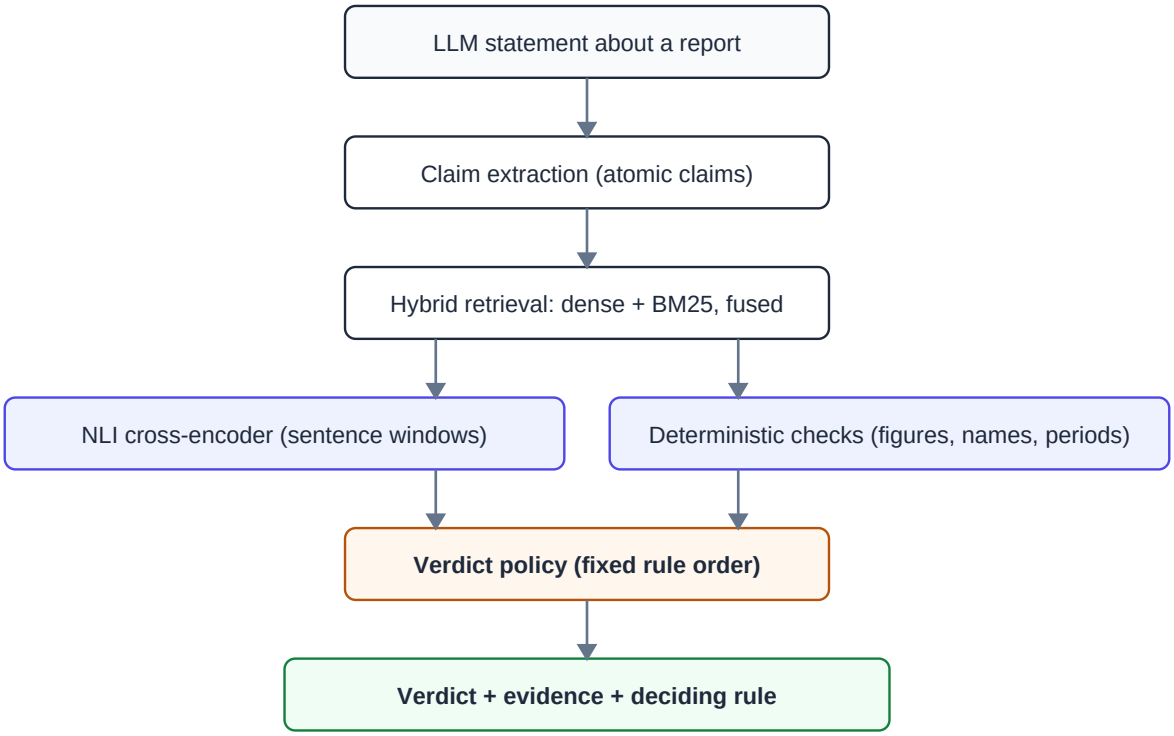


Figure 1. The verification pipeline. NLI and the deterministic checks run side by side; the verdict policy returns the first rule that fires.

3.1 The verification pipeline

- **Ingestion.** An uploaded PDF is validated (extension, type, magic bytes, size), hashed with SHA-256, split into page-aware chunks, and its tables are extracted with their units and periods. Scanned pages go through OCR when Tesseract is available.
- **Claim extraction.** The statement is split into atomic claims. Splits at abbreviations such as "N.A." are repaired, and each sub-claim keeps its subject and reporting period.
- **Hybrid retrieval.** Dense (MiniLM) and BM25 rankings are merged by reciprocal rank fusion and the top three chunks are kept. The right page is among them for 95.0% of test claims (mean reciprocal rank 0.885).
- **NLI.** The cross-encoder scores entailment and contradiction against sentence-sized windows of the evidence, and only the windows that align best with the claim are allowed to decide.

- **Deterministic checks.** Figure grounding (does the evidence state this figure, or a different one for the same fact), entity novelty (names that appear nowhere in the document), fiscal-period alignment, structured-table lookup, and recomputation of derived values such as growth and margins.
- **Verdict policy.** Applies the checks in a fixed order, `entity_novelty`, `figure_conflict`, `table_mismatch`, `temporal`, `multi_hop`, `nli`, `grounded_support`, and returns the first that fires. The winning rule is stored as *decided_by*. A statement's risk is that of its worst claim.

3.2 Service layout

Layer	Responsibility
API (`app/api`)	Versioned routes under `/api/v1`; the unversioned `/api` is kept and marked deprecated
Security	JWT and API-key authentication, roles (viewer, analyst, admin), tenant scoping, rate limits
Services	Ingestion, retrieval, NLI, numeric grounding, verdict policy, export, audit, job runner, analytics, benchmark
Repositories	The only code that touches the database; every query is scoped to the caller's organisation
Observability	Request IDs and W3C traceparent, JSON logs, Prometheus metrics
Frontend	React 19, TypeScript, Tailwind CSS, Framer Motion, Recharts; route guards by role

Verification runs as background jobs: submitting returns at once with a status URL, an in-process runner executes two jobs at a time with a bounded queue, and batches of up to 25 statements are accepted. Every record carries an organisation identifier and a request for another organisation's record answers 404, so identifiers cannot be probed.

4. Features built around the verifier

A verifier is only useful if people can run it, trust it and review its output. Around the core pipeline the application provides the following.

4.1 Accounts and access

Users sign in with a password, with a six-digit code sent to their email, or with Google. Registering with a password requires an emailed code, so an address cannot be claimed by someone who does not control it. A forgotten password is reset with an emailed code. Codes are random, stored only as a keyed hash, single-use, valid for ten minutes, locked after five wrong guesses and rate limited. A My Account page lets a user change their name and password and end every other session; changing or resetting a password ends all older sessions at once, because each token carries the account's session version. Roles are *viewer*, *analyst* and *admin*, read from the database on every request, and API keys never exceed their owner's role.

4.2 Evidence and review

Each verdict links to a claim page that shows the NLI scores, the numerical audit with the formula used, the temporal check and a plain-language diagnostic, and the evidence page can be opened in a PDF viewer. A review queue holds claims for a human decision. Results export as a PDF report (with a provenance block), CSV and a printable audit.

4.3 Audit trail and provenance

Each organisation has a hash chain of events: an event stores a sequence number, the previous event's hash and its own SHA-256 over its content, and the chain is verified on demand. Every session stores its provenance: the pipeline version, model identifiers, retrieval and policy settings, and the SHA-256 of the document and of the statement. Provenance holds identifiers and settings only, never document text or credentials.

4.4 Analytics and benchmark pages

The **Analytics** page summarises an organisation's finished verifications over 7, 30 or 90 days: totals with the change in hallucination rate against the previous period, claims per day by verdict, the documents and claim types most affected, the kinds of hallucination found, which rule decided each claim, and the flagged claims most worth a reviewer's time. The **Benchmark** page shows the recorded held-out evaluation results described in Section 6, read from the evaluation reports and never recomputed. In both, every chart can be shown as a table, the verdict colours were checked for colour-blind separation against the dark surface, and verdicts always carry an icon and a label as well as a colour.

5. Walkthrough

The screenshots below were taken from the running application. The Northstar sample is a synthetic seven-page annual report that ships with the application, together with a seven-sentence statement about it in which some claims are wrong.

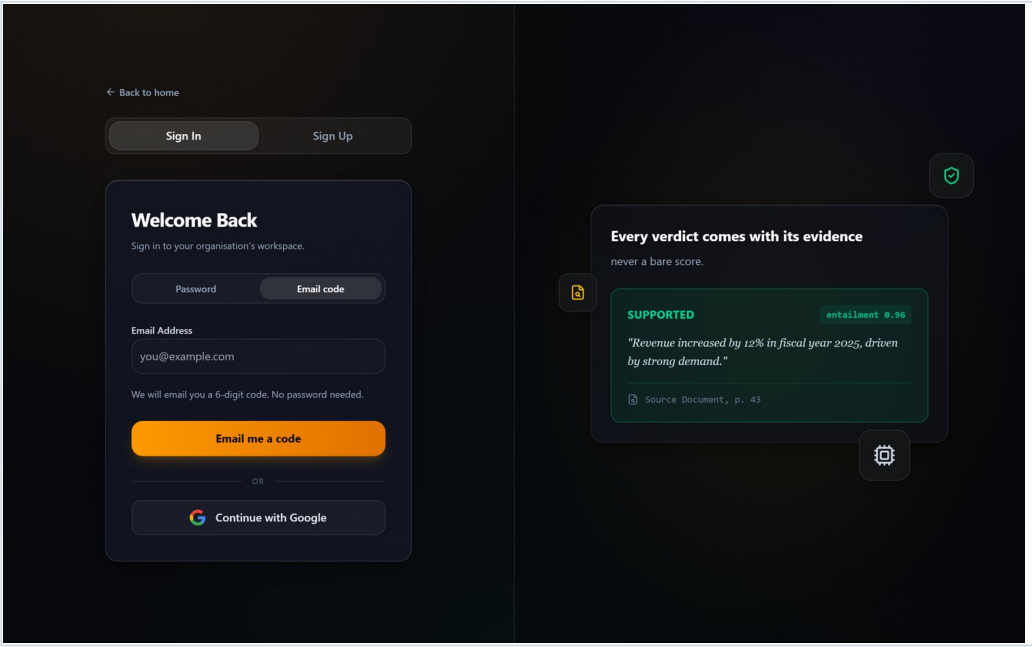


Figure 2. Sign-in with an emailed code, with the password and Google options alongside.

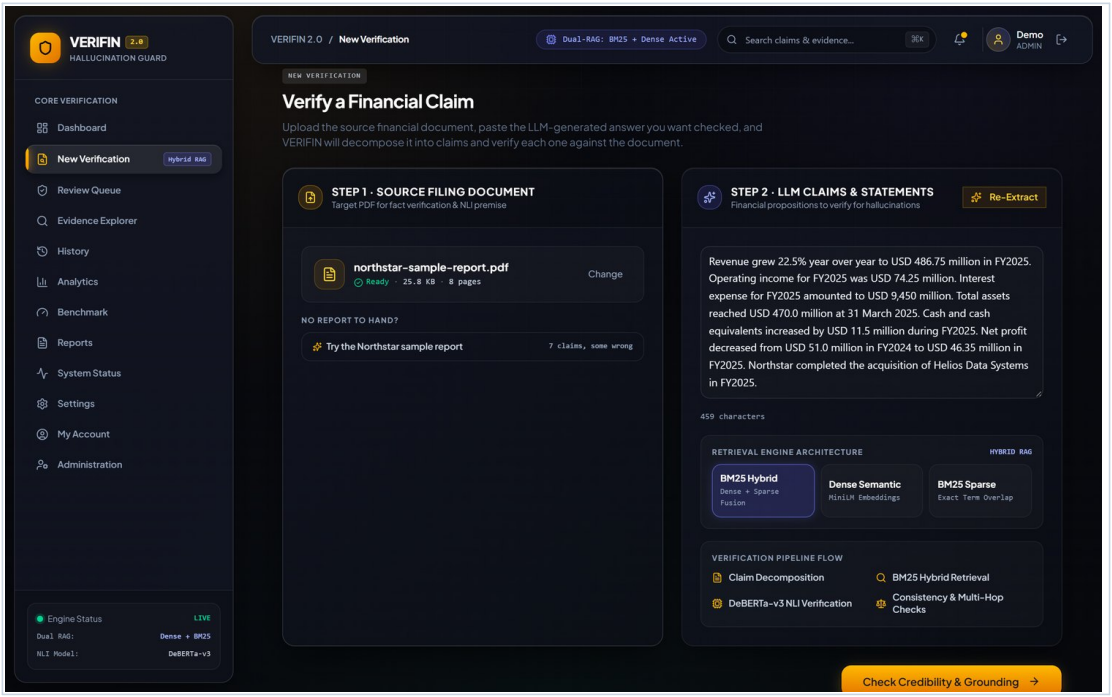


Figure 3. New Verification: the sample report has been uploaded and the statement filled in.

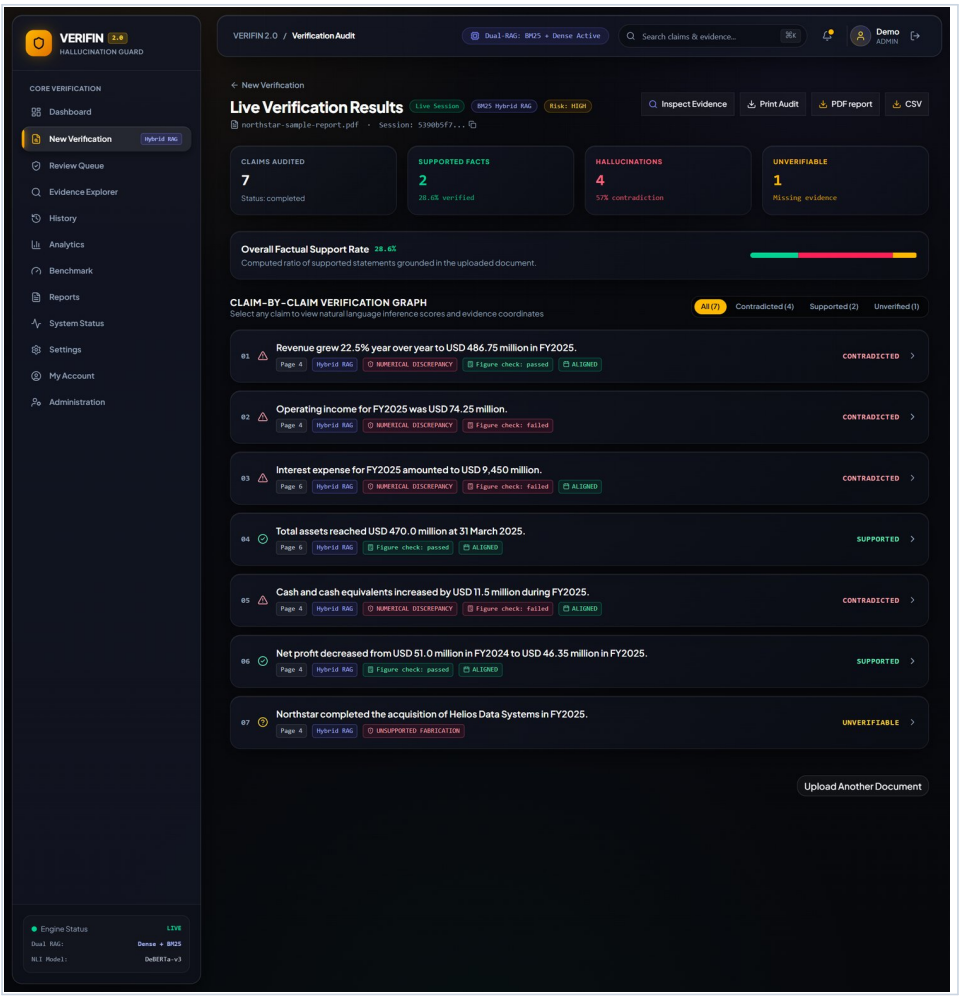


Figure 4. Results for the Northstar sample, produced by the real pipeline: four claims contradicted, two supported, one unverifiable, and the statement rated high risk.

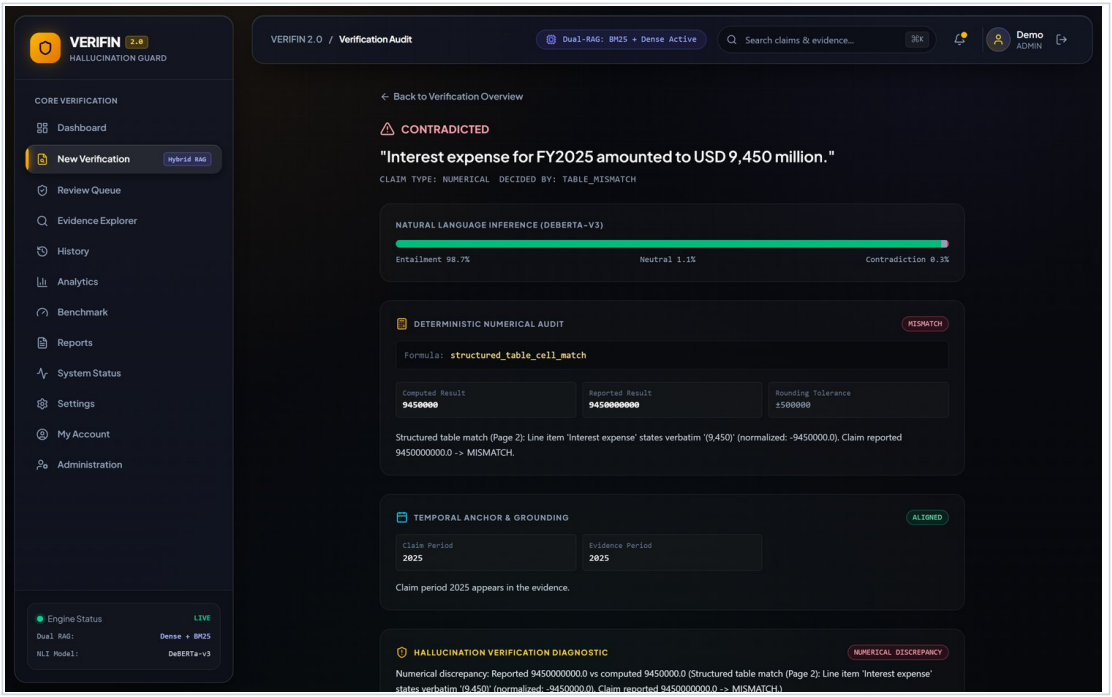


Figure 5. Claim detail for the interest-expense claim. The NLI model alone gives 98.7% entailment; the structured-table check finds that the report states 9.45 million, not 9,450 million, and the deciding rule is table_mismatch.

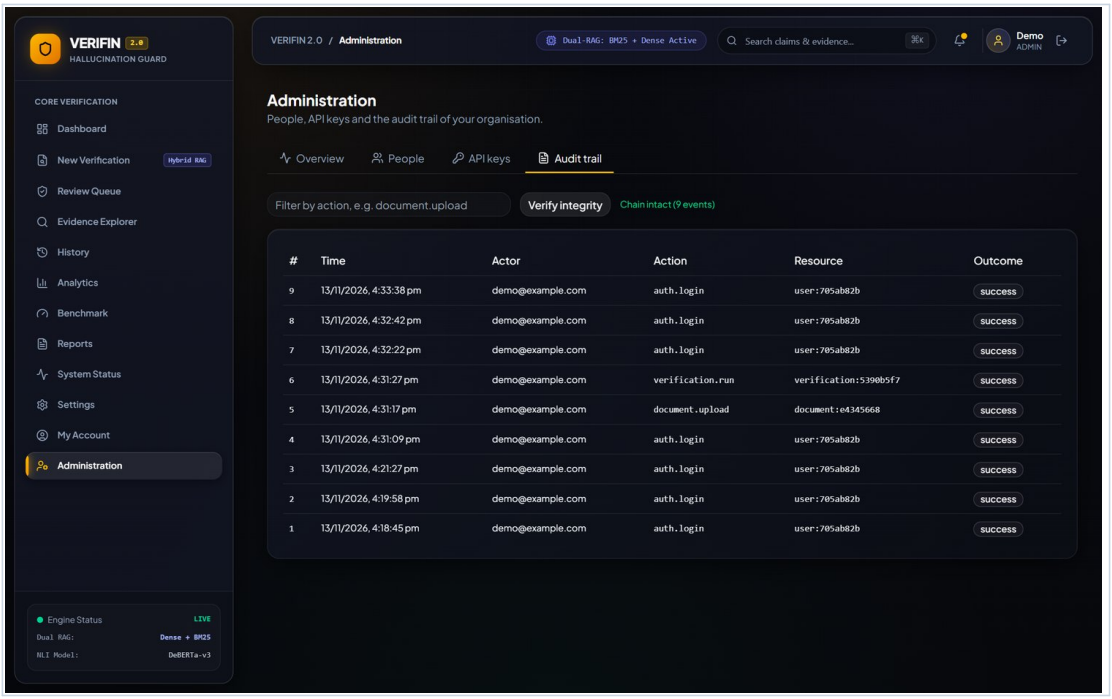


Figure 6. The audit trail with the integrity check run: "Chain intact" for nine events.

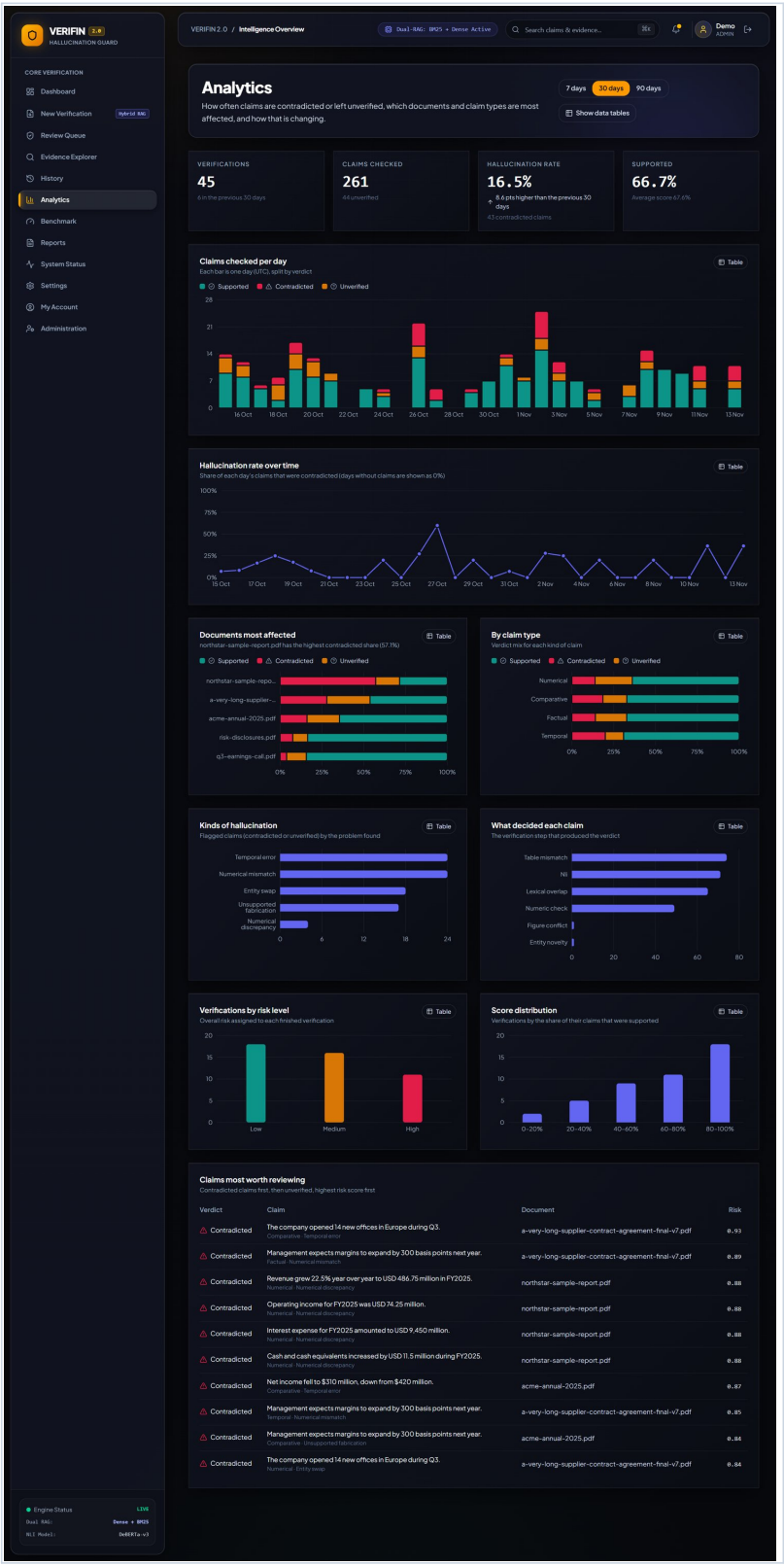


Figure 7. The Analytics page. ***This screenshot uses generated sample data*** *(44 synthetic verifications over 35 days, plus the one real Northstar run), not real usage; it shows the layout and charts.*

6. Evaluation

6.1 Data and method

The evaluation corpus is six public documents: the JPMorgan 2025, Reliance 2025-26, SBI 2025-26 and Tata Motors 2024-25 annual reports, an HDFC Bank AGM transcript and a Wipro AGM notice, about 16,000 text chunks. From them 594 claims were built so that each label is correct by construction:

- **Supported (282).** A verbatim report sentence (195), or a paraphrase by a 7B model accepted only if every number is preserved and no antonym of the original direction word is introduced (87).
- **Contradicted (212).** One controlled edit of a supported sentence: a changed number (115), a unit swap (34), a period swap (32) or a direction flip (31). A numeric edit is rejected if the new value appears anywhere in the report.
- **Unverifiable (100).** Plausible claims about invented entities (a penalty, facility, bond, acquisition or appointment) whose names appear nowhere in the report.

The split is by a hash of the document and page: 354 claims for development and 240 for test (109 supported, 91 contradicted, 40 unverifiable). All rules and thresholds were set on the development split. The test split was scored after the rules were frozen, and re-run once on the shipped pipeline as a regression check with no tuning in between; all 240 verdicts were identical. Metrics are accuracy and macro-F1 with 95% bootstrap intervals, an exact McNemar test against the reference system on the same claims, retrieval recall and mean reciprocal rank, expected calibration error and latency on a CPU.

6.2 Held-out results

System (test split, 240 claims)	Accuracy [95% CI]	Macro-F1 [95% CI]	s per claim
VERIFIN	**94.6** [91.7, 97.5]	**95.2** [92.3, 97.7]	1.0
VERIFIN, end-to-end with the real claim extractor	93.8 [90.4, 96.7]	94.2 [90.9, 97.0]	-
VERIFIN with the larger NLI model (deberta-v3-base)	93.3 [90.0, 96.2]	93.7 [90.3, 96.5]	1.6
NLI output only (no deterministic checks)	75.8 [70.0, 80.8]	66.8 [59.2, 73.0]	1.0
Number-match heuristic (no ML)	81.7 [76.2, 86.7]	79.7 [73.2, 85.1]	-
7B language-model judge, given the same evidence	83.8 [78.8, 87.9]	82.2 [76.8, 86.7]	12.4
7B language-model judge, no document	20.0 [15.0, 25.0]	14.6 [10.5, 18.6]	9.0

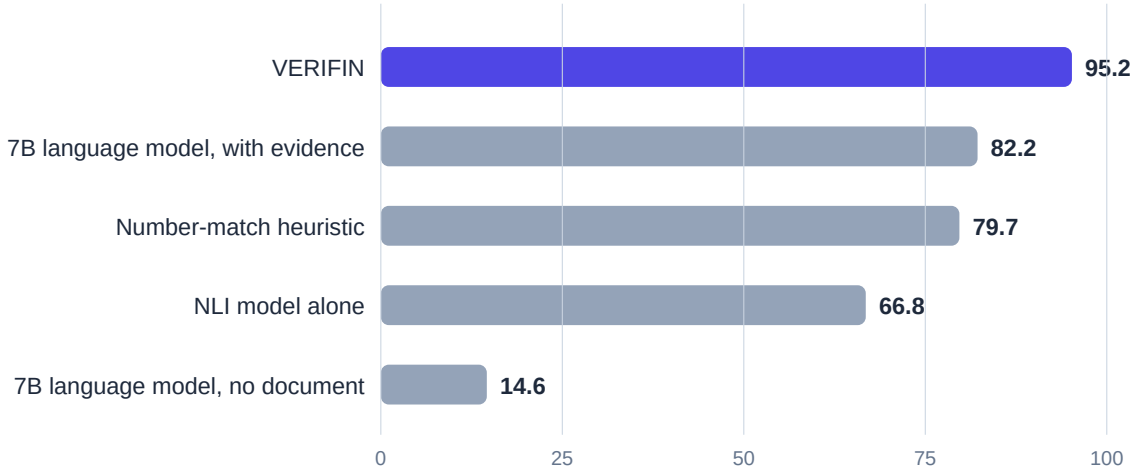


Figure 8. Macro-F1 on the held-out test split. The 7B judge without the document answers "unverifiable" for 231 of 240 claims, because it cannot know the report's figures.

Differences between VERIFIN, the end-to-end variant and the larger NLI model are not statistically significant (McNemar p = 0.63 and 0.38). The differences against the NLI-only system, the heuristic and both language-model judges are ($p < 0.001$). Treating any non-supported verdict as a flag for review gives precision 93.4%, recall 97.7% and a false-positive rate of 8.3%.

6.3 Where the errors are

True \ said	Supported	Contradicted	Unverifiable	Claims
Supported	**100** (92%)	**8** (7%)	**1** (1%)	109
Contradicted	**3** (3%)	**88** (97%)	**0** (0%)	91
Unverifiable	**0** (0%)	**1** (2%)	**39** (98%)	40

Table: confusion matrix on the test split. Rows are the true label, columns what VERIFIN said; each cell shows the count and its share of the row.

Class	Precision	Recall	F1	Claims
Supported	97.1%	91.7%	94.3	109
Contradicted	90.7%	96.7%	93.6	91
Unverifiable	97.5%	97.5%	97.5	40

On the development split by error type, a changed number is caught 98.4% of the time, a period swap 100%, a direction flip 95.7% and a unit swap 94.7%; verbatim supported claims score 95.7% and paraphrased ones 91.1%. On test the weakest areas were paraphrased supported claims (83.9%) and unit swaps (93.3%). Calibration is poor: the expected calibration error is 0.183, so the confidence value is an evidence-alignment score and not a probability.

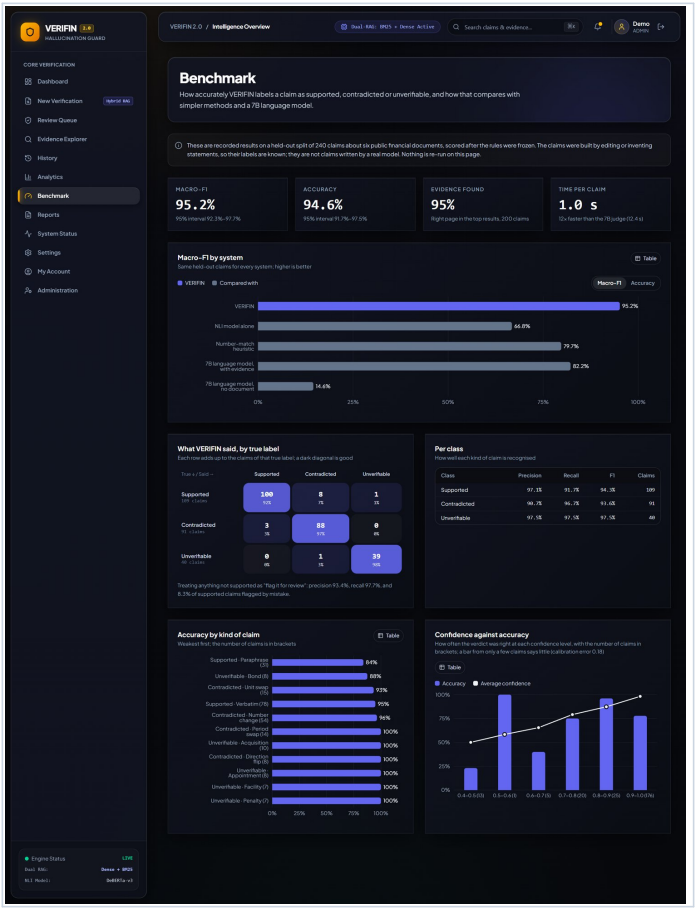


Figure 9. The Benchmark page, which charts these recorded results.

6.4 What produced the gain

The pipeline improved in steps on the development split. The original system scored a macro-F1 of 29.4, worse than the number-match heuristic, mainly because its numerical reasoner compared a claim with arbitrary numbers in the evidence (footnotes, page numbers) and falsely flagged 80% of supported claims.

Version (dev split, 354 claims)	Accuracy	Macro-F1 [95% CI]
v0 original system	42.7	29.4 [25.7, 32.9]
v1 + figure grounding, sentence-window NLI, verdict policy	92.9	93.9 [91.3, 96.1]
v3 + local-context grounding, neighbour chunks, fiscal-year check, unit-twin rule	96.0	96.1 [93.7, 98.0]
v4 (current) + multi-figure support rule	**96.3**	**96.7** [94.5, 98.3]

A leave-one-out ablation of the current pipeline switches off one check at a time (development split, p is McNemar against the full pipeline):

Check switched off	Macro-F1	Change	p
none (full pipeline)	96.7	-	-
Entity novelty (names absent from the document)	76.4	-20.3	< 0.001
Figure conflict (a changed figure in the matching sentence)	92.7	-4.0	< 0.001
Grounded support (figures all found, NLI neutral)	93.9	-2.8	0.016

Check switched off	Macro-F1	Change	p
Fiscal-year period check	95.2	-1.5	0.031
Neighbour-chunk expansion	95.6	-1.1	0.625 (not significant)
Derived-value recomputation; table arithmetic	96.7	0.0	1.0

Four checks make a significant difference. The last row shows no effect because the benchmark has no derived-percentage claims and no structured tables; those checks are covered by unit tests, not by this evaluation, and are not credited with the measured accuracy.

6.5 A table-heavy probe

The benchmark is built from sentences of narrative reports, so it cannot show how the system handles financial statements laid out as tables. To look at this, 24 claims about the synthetic Northstar report were labelled by reading its figures: 11 management-summary highlights (some contradicting the statements by design), 8 true restatements of table rows and 5 edited claims. VERIFIN labels 21 of 24 correctly (87.5%). The three errors are a false alarm on a true two-figure claim, a true percentage claim misread by NLI, and a claim whose figures are right but whose direction is wrong ("net profit increased by 9.1%" when it fell). The probe guided bug fixes, so 87.5% is a development figure with a wide interval (roughly 68% to 97%), not a held-out result.

7. Security and privacy

The threat model follows the usual order: assets, trust boundaries, threats, controls, and the risk that remains. It is a design-level assessment by the authors; no independent penetration test has been done. The main threats and controls are summarised below; the full table has seventeen entries.

Threat	Control	Risk that remains
Reading another organisation's data	Organisation identifier on every record; repositories filter on it; cross-organisation access answers 404; tests cover it	A code path that forgets the filter would leak; nothing enforces the filter structurally
Forged or stolen tokens	HS256 JWT with fixed algorithm and required claims; a session version in each token ends all of an account's tokens at once	One shared signing secret; a single token cannot be revoked before it expires except by ending the account's sessions
Credential guessing	scrypt hashing, minimum length, dummy hash for unknown emails, rate-limited sign-in	Limits are per process and per address
Abuse of emailed codes, forged Google sign-in	Random single-use codes stored as keyed hashes, expiry, attempt limits; Google tokens verified server-side	A mailbox takeover is a sign-in takeover
Malicious upload	Extension, type, magic-byte and size checks; generated file names; non-root container	Parsers process untrusted files; mitigated by dependency scanning
Prompt injection through a document	No language model is used in verification, so document text cannot change behaviour	The optional fact-extraction endpoint does send text to a hosted model
Tampering with the audit trail	Per-organisation SHA-256 hash chain verified on demand; unique sequence index	A database administrator could rewrite the whole chain
Denial of service	Per-address and per-user rate limits, a bounded queue, a 25 MB upload limit	Limits are in memory and per instance

Disclosed gaps include no Content-Security-Policy header, no multi-factor authentication beyond the mailbox, no account deletion in the application, and a design that assumes a single process. The verification path itself is local: documents are not sent to any outside service while a statement is checked.

8. Testing and engineering practice

- **Automated tests.** The backend has 374 passing tests (one skipped), covering the pipeline rules, organisation scoping, authentication, rate limits, the audit chain, reports, analytics and the benchmark summary.
- **Static checks.** Ruff for style, Bandit for security patterns, and a type check of the frontend.
- **Continuous integration.** On every push CI runs secret scanning (gitleaks), the backend suite, the frontend build and checks, and builds the container image and scans it for known vulnerabilities. At the time of writing the image vulnerability scan is failing on a newly reported advisory that has not yet been identified; the other jobs pass.
- **Reproducibility.** Every verdict records the pipeline version, models and settings; the evaluation can be re-run from the command line (Appendix B), and raw per-claim predictions are stored with the results.
- **Honest reporting.** Numbers on the development split are labelled as optimistic, the test split was scored once, and results that were not significant are said to be so.

9. Limitations and responsible use

- **Tables and statement layouts** are the weakest area; the benchmark has no structured-table claims, and the probe scored 21 of 24.
- **Direction claims** with correct figures can be missed when no sentence states the direction.
- **"Unverifiable" is narrow.** The 100% score on that class comes from a rule that flags names absent from the document. Plausible facts that the report simply does not state, and that use no new names, are not covered.
- **The benchmark is synthetic.** Contradictions are programmatic edits, mostly numeric, and labels have not been audited by humans. The results measure the method on known errors, not claims written by a real model.
- **Calibration.** Confidence is not a probability (expected calibration error 0.183).
- **The language-model comparison** is one small local model at temperature 0 with an untuned prompt; it shows what a 7B model does with this evidence, not what language-model judging can do.
- **Scope.** English reports only; Indian digit grouping, crore and lakh are handled and other number conventions are untested; OCR accuracy has not been measured.

The main risk in use is over-trust: a green "supported" badge on a statement the system got wrong. The interface therefore shows the evidence and the deciding rule, uses "unverified" instead of forcing a guess, and records what was shown. A "supported" verdict should be read as "no problem found", not as "confirmed". The system has no personal attributes as inputs.

10. Future work

- Audit a sample of 40 to 50 benchmark labels by hand and report inter-rater agreement.
- Add structured-table claims, derived-percentage claims and subtle semantic contradictions to the benchmark, so the table and arithmetic checks can be measured.

- Check the direction of change against table rows, which would address the one missed error in the probe.
- Measure OCR quality on scanned reports, and test other number conventions and languages.
- Improve calibration so that the confidence value can be read as a probability.
- Add a Content-Security-Policy header and multi-factor authentication, and move the rate limiter and job queue to shared storage so that more than one instance can run.

11. Conclusion

VERIFIN 2.0 shows that most of the accuracy in checking a language model's financial claims against a report can come from a transparent, deterministic layer around a small NLI model, and not from a larger model. On 240 held-out claims it reaches a macro-F1 of 95.2 at about one second per claim on a CPU, against 82.2 at twelve seconds for a 7B language-model judge given the same evidence. Every verdict carries its evidence page and deciding rule, every action is in a verifiable audit trail, and the limits of the evidence are stated alongside the results. It is a screening aid that tells an analyst where to look first, and it is meant to be used that way.

Appendix A. Main API endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/login, /register, /me	Sign in, register (with an emailed code) and read the current user
POST	/api/v1/auth/password/reset/request and /confirm	Reset a password with an emailed code
POST	/api/v1/auth/sessions/revoke	End every other session of the account
POST	/api/v1/documents/upload	Upload and ingest a PDF
POST	/api/v1/verification/jobs	Queue a verification (202 with a status URL)
GET	/api/v1/verification/{id}/results	Session, claims, verdicts and evidence
GET	/api/v1/analytics/overview?days=30	Aggregate statistics for the Analytics page
GET	/api/v1/benchmark/summary	Recorded held-out evaluation results
GET	/api/v1/audit/verify	Recompute and check the audit hash chain (admin)
GET	/api/v1/system/health	Service and database status

Appendix B. Reproducing the results

From the `backend` directory:

```
python -m eval.build_dataset # rebuild the dataset (needs a local 7B model)
python -m eval.run --split dev --systems verifin --run v4
python -m eval.report --split dev --systems verifin --run v4
python -m pytest # backend tests
python ../scripts/probe_report.py http://127.0.0.1:8000 # 24-claim table probe
```

Raw per-claim predictions are stored under `data/eval/results/<run>/<split>/`. The application runs locally with `scripts/start_demo.ps1`. Further documents in the repository: the model card, architecture, threat model, runbook, deployment notes and the demo script.

Appendix C. Notes on the figures

Figures 2 to 6 were captured from the running application on the Northstar sample, using the real pipeline (version 4.2). Figure 7 (Analytics) was captured with generated sample verifications added so that the charts have something to show, and it is the only screenshot that does not reflect real use. Figure 9 (Benchmark) charts the recorded evaluation results, which are read from the evaluation reports and not recomputed.